HADOOP

Comprehensive Study Notes

HDFS • MapReduce • YARN • Ecosystem

Hadoop Distributed File System | MapReduce | Administration

1. Introduction to Hadoop

1.1 What is Hadoop?

Apache Hadoop is an open-source framework for distributed storage and processing of massive datasets across clusters of commodity hardware. It was developed by Doug Cutting and Mike Cafarella, inspired by Google's seminal papers on the Google File System (GFS) and MapReduce. Written in Java, Hadoop is part of the Apache ecosystem and is designed to scale from a single server to thousands of machines, each offering local computation and storage.

Core Goal: Process huge volumes of data efficiently across clusters of commodity hardware — reliably, cost-effectively, and in parallel.

1.2 Why Hadoop?

Traditional RDBMS systems break down when confronted with Big Data challenges:

- Volume – Terabytes to petabytes of data overwhelm single-node systems.
- Variety – Structured, semi-structured, and unstructured data from many sources.
- Velocity – Real-time or near-real-time ingestion and processing needs.
- Scalability limits – Vertical scaling (bigger servers) is expensive and has a ceiling.
- High storage cost – Enterprise storage solutions are cost-prohibitive at scale.

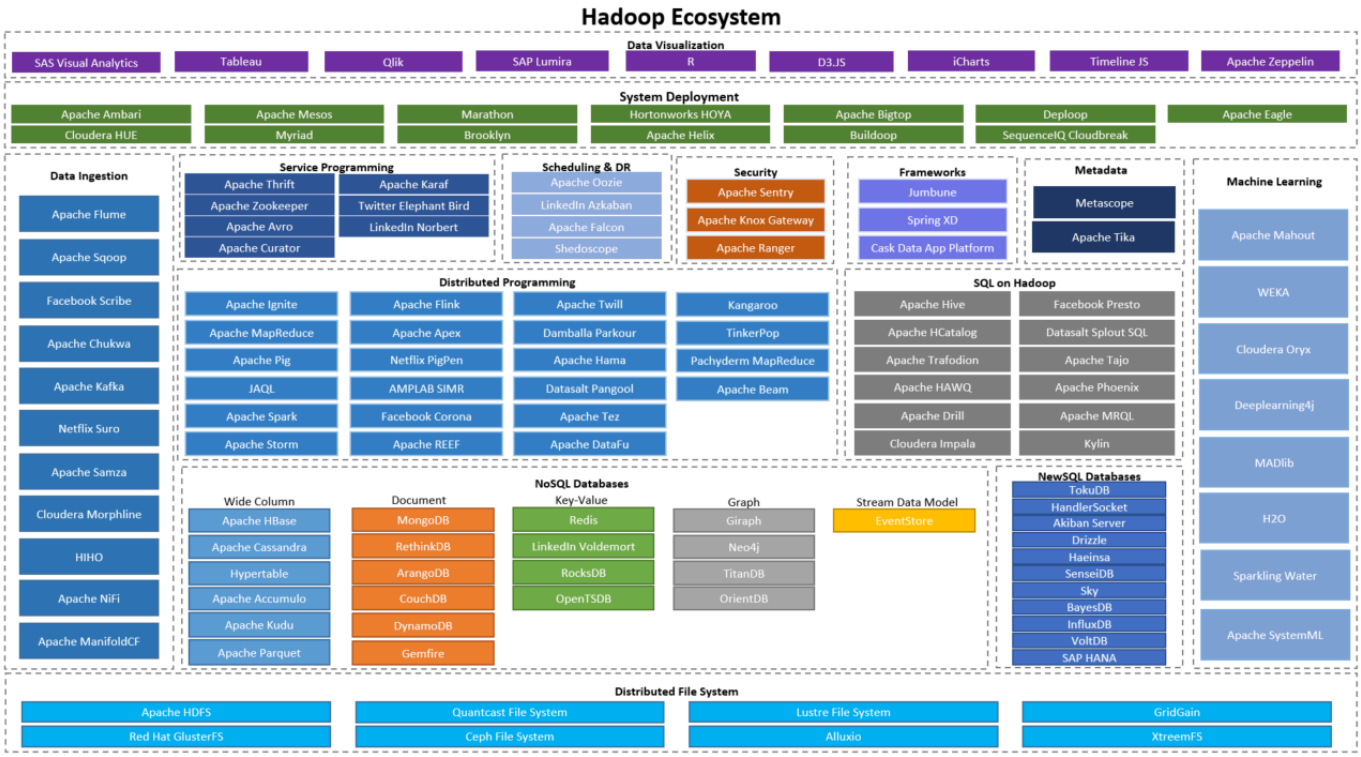
1.3 Core Components of Hadoop

Component	Description
HDFS (Hadoop Distributed File System)	Storage layer – stores large files across multiple machines in a fault-tolerant manner.
YARN (Yet Another Resource Negotiator)	Cluster management layer – handles resource allocation and job scheduling for all workloads.
MapReduce	Data processing layer – programming model for distributed, parallel computation.
Common Utilities	Libraries and Java utilities that support other Hadoop modules.

1.4 Hadoop Architecture Overview

Hadoop follows a Master-Slave architecture at every layer:

- HDFS → NameNode (Master), DataNode (Slave)
- MapReduce → JobTracker (Master), TaskTracker (Slave) [Hadoop 1.x]
- YARN → ResourceManager (Master), NodeManager (Slave) [Hadoop 2.x+]



Data is split into fixed-size blocks and replicated across nodes for redundancy.

Version	Default Block Size	Key Feature
Hadoop 1.x	64 MB	Single NameNode, basic MapReduce
Hadoop 2.x	128 MB	YARN introduced, NameNode HA
Hadoop 3.x	256 MB	Erasure Coding, multiple Standby NameNodes

1.5 Hadoop Ecosystem Components

Hadoop is surrounded by a rich ecosystem of complementary tools:

Tool	Purpose
Pig	Scripting platform for large-scale data processing using Pig Latin.
Hive	SQL-like data warehouse system; translates HiveQL to MapReduce/Tez/Spark.
HBase	NoSQL column-family database built on top of HDFS.
Sqoop	Transfers data between Hadoop and relational databases (RDBMS).
Flume	Ingests streaming log data into HDFS reliably.

Tool	Purpose
Oozie	Workflow scheduler for managing Hadoop jobs.
ZooKeeper	Distributed coordination service (used for NameNode HA failover).
Spark	In-memory big data processing engine; much faster than MapReduce.

2. HDFS — Hadoop Distributed File System

2.1 Introduction to HDFS

HDFS (Hadoop Distributed File System) is a scalable, fault-tolerant, distributed storage system designed to store and manage large datasets across clusters of commodity hardware. It was inspired by the Google File System (GFS) paper published in 2003.

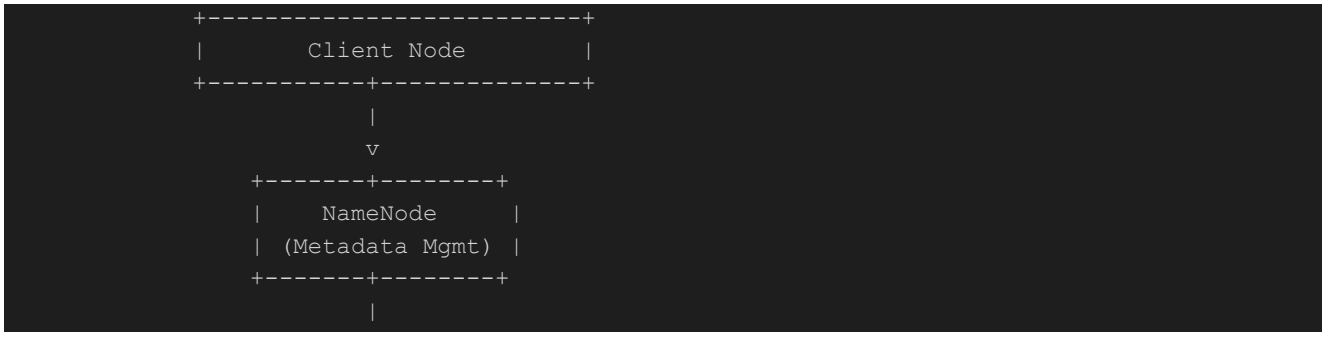
2.3 HDFS Architecture

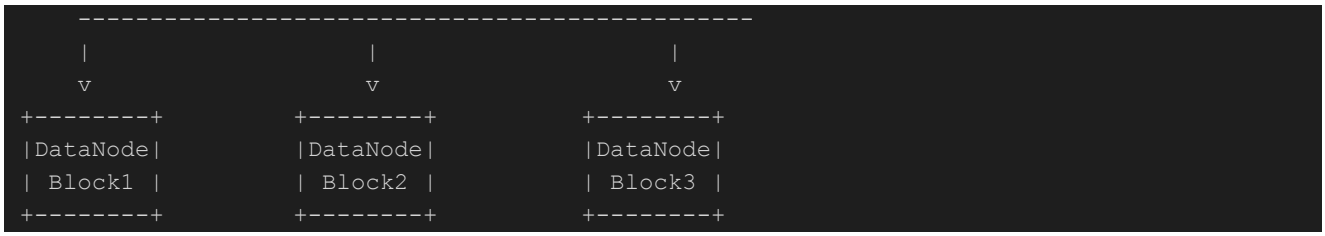
Master-Slave Design

Component	Role	Details
NameNode (Master)	Manages the file system namespace and metadata	Maintains directory structure, permissions, file-to-block and block-to-DataNode mappings. Stores FsImage + EditLog on disk; entire namespace kept in RAM.
DataNode (Slave)	Stores actual data blocks	Handles read/write requests from clients. Sends heartbeats every 3 seconds and periodic block reports to NameNode.
Secondary NameNode	Checkpoint helper	NOT a backup — periodically merges FsImage and EditLog to create checkpoints, preventing EditLog from growing too large.

⚠️ **Important: The Secondary NameNode is NOT a hot standby. It does not take over if the primary NameNode fails. For true fault tolerance, configure NameNode High Availability (Active/Standby) using ZooKeeper and JournalNodes.**

HDFS Architecture Diagram (Conceptual)





2.4 How a File is Stored in HDFS

Example: A 512 MB file with 128 MB block size and replication factor 3:

- File is split into 4 blocks of 128 MB each.
- Each block is replicated to 3 different DataNodes.
- Total storage consumed = 4 × 3 = 12 block copies.
- NameNode records which DataNode holds which replica.
- If any DataNode fails, HDFS auto-creates new replicas to maintain the replication factor.

Customer data example: customer_data.csv (300 MB), block size 128 MB, replication factor 3:

- Block 1: 128 MB → replicated to DataNodes A, B, C
- Block 2: 128 MB → replicated to DataNodes B, C, D
- Block 3: 44 MB → replicated to DataNodes A, C, D
- Total storage used: 300 MB × 3 = 900 MB

2.5 HDFS Read and Write Workflow

Write Operation

- Client contacts NameNode → requests file creation.
- NameNode checks namespace and grants permission.
- File is split into blocks (e.g., 3 blocks for a 300 MB file).
- NameNode assigns DataNodes for each block.
- Client writes blocks → DataNodes store and replicate them automatically.
- After successful write, NameNode updates its metadata.

Read Operation

- Client asks NameNode for block locations of the requested file.
- NameNode responds with the list of DataNode addresses per block.
- Client reads data directly from the nearest DataNodes.
- File is reconstructed in order (Block 1 → Block 2 → Block 3).

2.6 What is Stored Inside the NameNode

The NameNode stores metadata — information about the data — not the actual data itself. All metadata is kept in RAM for fast access.

Type of Information	Description
File & Directory Structure	Hierarchical namespace — directory tree of all files and folders in HDFS (inode-based).
File-to-Block Mapping	Which file is split into which blocks (e.g., /user/data.txt → block_001, block_002…).
Block-to-DataNode Mapping	Which DataNodes store each block (e.g., block_001 → DataNode 3, 7, 8).
Replication Information	Replication factor for each block (default: 3).
Permissions & Ownership	POSIX-style permissions: owner, group, read/write/execute flags.
Timestamps	File creation, modification, and access times.
Quota Information	Directory-level quotas: max number of files or total storage allowed.
Namespace ID & Cluster ID	Unique identifiers to distinguish HDFS clusters and namespaces.
FsImage + EditLog	Persistent metadata storage on disk — FsImage is a full snapshot; EditLog records recent changes.

Memory Requirement: The NameNode keeps all metadata in RAM for fast access. A rough guideline is ~1 GB of RAM per 1 million files/blocks. This is why Hadoop 1.x faced scalability limits with very large clusters.

Files Maintained by the NameNode

File	Purpose
FsImage	Snapshot of the entire HDFS namespace at a point in time (like a full metadata backup). Loaded into RAM on startup.
EditLog	Transaction log recording all recent HDFS changes (create, delete, rename, etc.). Applied on top of FsImage at startup.

During startup: NameNode loads FsImage into memory, then applies all changes from EditLog to reconstruct the current state of the filesystem.

2.7 HDFS Namespaces

A namespace in Hadoop is the hierarchical organization of files and directories in HDFS — essentially the directory tree structure holding all metadata about files and directories.

Components of a Namespace

NameNode — The master node that manages the namespace. It stores and maintains all metadata (file names, directory structure, permissions, block mappings) in memory for fast access. It does not store actual data, only the metadata.

Namespace Metadata — The actual directory tree data, which includes:

FsImage — A persistent snapshot of the entire namespace stored on disk. It captures the complete state of the file system at a point in time.

EditLog — A transaction log that records every change (create, delete, rename) made to the namespace since the last FsImage checkpoint.

Block Mapping — The association between files and their physical data blocks stored across DataNodes. Stored in the NameNode's memory but rebuilt from DataNode reports on startup.

Secondary NameNode — A helper node that periodically merges the FsImage and EditLog to create an updated checkpoint, preventing the EditLog from growing too large. (Note: It is NOT a backup NameNode.)

Namespace ID — A unique identifier assigned to an HDFS instance at format time. All DataNodes must register with this ID to join the cluster.

Inodes — Internal data structures representing each file and directory in the namespace, holding attributes like permissions, timestamps, replication factor, and block list.

Namespace Quotas

Quota Type	Description	Command
Name Quota	Limits the number of file/directory names in the tree.	hdfs dfsadmin -setQuota <N> <directory>
Space Quota	Limits the amount of disk space used by a directory.	hdfs dfsadmin -setSpaceQuota <N> <directory>

HDFS Federation (Multiple Namespaces)

Hadoop 2.x+ supports HDFS Federation where multiple independent NameNodes each manage their own namespace volume. This brings:

- Scalability – namespace is no longer a single bottleneck.
- Isolation – one namespace's load does not affect others.
- Increased throughput – parallel metadata operations.

2.8 Version-wise HDFS Evolution

Feature	HDFS 1.x	HDFS 2.x	HDFS 3.x
Architecture	Single NameNode (SPOF)	Federation: Multiple NameNodes	Enhanced HA with multiple Standby NNs
High Availability	No HA	Active/Standby NameNodes	Improved failover and auto recovery
Federation	Not available	Supported	Improved scalability
Storage Efficiency	3× replication only	3× replication	Erasure Coding (~1.5× overhead)
Default Block Size	64 MB	128 MB	256 MB
YARN Integration	Not present	Introduced	Optimized
Container/Docker	Not supported	Limited	Fully supported
Fault Recovery	Manual restart	Automatic failover	Advanced recovery mechanisms

HA Setup (Hadoop 2.x/3.x): One Active NameNode handles all client requests. One or more Standby NameNodes stay synchronized via JournalNodes. If the Active NN fails, ZooKeeper automatically promotes a Standby NN to Active.

2.9 Replication vs. Erasure Coding

Hadoop offers two methods to protect data from failures. Choosing between them depends on data access patterns.

Traditional Replication (Hadoop 2 & 3)

Hadoop's default method — makes three full copies of every block.

- Storage cost: 3 TB for every 1 TB of data (200% overhead).
- Fault tolerance: Can lose any 2 copies and data is still safe.
- CPU usage: Very low — simple file copies.
- Best for: 'Hot' data — frequently accessed and frequently modified files.

Erasure Coding (Hadoop 3)

Uses a 'data + parity' model. With the common RS-6-3 policy:

- A 1 TB file is split into 6 data blocks.
- The system computes 3 parity blocks.
- All 9 blocks are stored on 9 different nodes.
- Storage cost: ~1.5 TB for 1 TB of data (only 50% overhead).
- Fault tolerance: Can lose any 3 of 9 blocks and reconstruct data.
- CPU usage: High — complex encoding/decoding math on every read/write.
- Best for: 'Cold' data — large, infrequently accessed archives or backups.

Feature	Replication (Hadoop 2 & 3)	Erasure Coding (Hadoop 3)
Storage Cost	Very High (200% overhead)	Low (~50% overhead)
CPU Usage	Very Low (simple copies)	High (complex math)
Network Usage	Low for reads, high for writes	High (rebuilding needs multiple blocks)
Recovery Speed	Fast (copy one block)	Slower (decode from multiple blocks)
Supports Append	Yes	No
Best For	Hot, frequently accessed data	Cold/warm archives and backups

EC Limitations: Higher CPU load, slower recovery, and does NOT support file operations like append or hflush.

2.10 HDFS vs Traditional File Systems

Feature	Traditional FS (NTFS/ext4)	HDFS
Storage Type	Local (single system)	Distributed across many nodes
Scalability	Limited to single disk/server	Horizontally scalable (add nodes)
Fault Tolerance	Hardware failure = data loss	Auto block replication prevents loss
Data Size	Best for small-medium files	Optimized for large files (GBs–TBs)
Access Speed	Low latency for small files	High throughput for large data
Write Model	Random read/write supported	Write-once, read-many
Replication	Manual backup required	Automatic, managed by NameNode
Hardware	Expensive, reliable servers	Commodity, low-cost hardware
Metadata	Stored locally on disk	Centralized in NameNode (RAM)
Usage	OS-level file storage	Big Data analytics platforms

2.11 Java Interfaces to HDFS

Key classes in the org.apache.hadoop.fs package:

- FileSystem – abstract base class for all filesystem operations.
- Path – represents a file or directory path in HDFS.
- FSDataInputStream / FSDataOutputStream – streams for reading and writing data.

Common operations available:

- create(), open(), delete(), rename(), mkdirs()
- listStatus(), getFileStatus(), getFileBlockLocations()

2.12 HDFS Command Line Interface

Command	Purpose	Example
hdfs dfs -ls /	List files and directories	hdfs dfs -ls /user/hadoop/data/
hdfs dfs -mkdir /dir	Create a directory	hdfs dfs -mkdir -p /user/hadoop/output/
hdfs dfs -put <filename>	Upload local file to HDFS	hdfs dfs -put sales.csv /user/hadoop/data/
hdfs dfs -get <filename>l	Download file from HDFS	hdfs dfs -get /user/hadoop/result.txt
hdfs dfs -cat /path	Display file contents	hdfs dfs -cat /user/hadoop/sample.txt
hdfs dfs -rm -r /path	Delete directory recursively	hdfs dfs -rm -r /user/hadoop/temp/

Command	Purpose	Example
hdfs dfs -du -h /path	Show disk usage	hdfs dfs -du -h /user/hadoop/
hdfs dfs -setrep -w 2 /path	Set replication factor	hdfs dfs -setrep -w 5 /critical/data.txt
hdfs dfsadmin -report	Cluster health report	Shows live/dead nodes, capacity, usage
hdfs dfs -chmod 755 /file	Change permissions	hdfs dfs -chmod -R 755 /user/hadoop/dir
hdfs dfs -chown user:grp /file	Change ownership	hdfs dfs -chown hadoop:hadoop /file
hdfs dfs -cp /src /dst	Copy within HDFS	hdfs dfs -cp /source/file /destination/
hdfs dfs -mv /src /dst	Move/rename files	hdfs dfs -mv /old/path /new/path
hdfs dfs -tail /path	View last 1 KB of file	hdfs dfs -tail /user/hadoop/log.txt
hdfs dfs -df -h /	Show cluster disk space	hdfs dfs -df -h /

2.13 HDFS Administration Commands

dfsadmin Commands

Command	Purpose
hdfs dfsadmin -report	Show cluster status: live/dead nodes, capacity, usage.
hdfs dfsadmin -safemode enter	Manually enter safe mode (no writes allowed).
hdfs dfsadmin -safemode leave	Exit safe mode.
hdfs dfsadmin -safemode get	Check current safe mode status.
hdfs dfsadmin -safemode wait	Wait for safe mode to exit (useful in scripts).
hdfs dfsadmin -saveNamespace	Save current namespace to FsImage.
hdfs dfsadmin -refreshNodes	Refresh the list of DataNodes (after decommissioning).
hdfs dfsadmin -setQuota 1000 /dir	Set name quota of 1000 files for the directory.
hdfs dfsadmin -setSpaceQuota 10g /dir	Set space quota of 10 GB for the directory.
hdfs dfsadmin -clrQuota /dir	Clear name quota on a directory.
hdfs dfsadmin -clrSpaceQuota /dir	Clear space quota on a directory.

HDFS File System Check (fsck)

Command	Purpose
hdfs fsck /	Check overall file system health.
hdfs fsck /user/hadoop	Check health of a specific directory.
hdfs fsck /user/hadoop -files -blocks -locations	Detailed block information with DataNode locations.
hdfs fsck / | grep CORRUPT	Find all corrupt files in HDFS.

2.14 HDFS Web Interfaces

Interface	URL / Port	Information Shown
NameNode Web UI	http://namenode:9870 (Hadoop 3.x)	Cluster health, live/dead DataNodes, filesystem browser, capacity, block distribution.
DataNode Web UI	http://datanode:9864	Local block storage status, disk usage.

3. Hadoop Commands: HDFS vs Linux

3.1 File Operations Comparison

Operation	Linux Command	Hadoop Command
List files	ls -l	hdfs dfs -ls
List recursively	ls -R	hdfs dfs -ls -R
Create directory	mkdir dirname	hdfs dfs -mkdir /dirname
Create nested dirs	mkdir -p dir/subdir	hdfs dfs -mkdir -p /dir/subdir
Remove file	rm filename	hdfs dfs -rm /filename
Remove directory	rm -r dirname	hdfs dfs -rm -r /dirname
Copy file	cp source dest	hdfs dfs -cp /source /dest
Move/Rename	mv source dest	hdfs dfs -mv /source /dest
View file content	cat filename	hdfs dfs -cat /filename
View end of file	tail filename	hdfs dfs -tail /filename
View beginning	head filename	hdfs dfs -head /filename
Upload file	cp /local /remote	hdfs dfs -put /local /hdfs/
Download file	cp /remote /local	hdfs dfs -get /hdfs/file /local/
Append to file	cat file >> existing	hdfs dfs -appendToFile local /hdfs/file

3.2 File Information Comparison

Operation	Linux Command	Hadoop Command
File statistics	stat filename	hdfs dfs -stat /filename
Disk usage	du -h	hdfs dfs -du -h
Disk free space	df -h	hdfs dfs -df -h

Operation	Linux Command	Hadoop Command
Word/file count	wc filename	hdfs dfs -count /filename
File checksum	md5sum filename	hdfs dfs -checksum /filename

3.3 Permission Operations Comparison

Operation	Linux Command	Hadoop Command
Change permissions	chmod 755 file	hdfs dfs -chmod 755 /file
Recursive chmod	chmod -R 755 dir	hdfs dfs -chmod -R 755 /dir
Change owner	chown user:group file	hdfs dfs -chown user:group /file
Change group	chgrp group file	hdfs dfs -chgrp group /file

3.4 Key Differences: Linux vs Hadoop

Aspect	Linux File System	Hadoop HDFS
Storage Architecture	Single machine, direct disk access	Distributed across nodes; data split into 128 MB blocks, replicated 3×
Command Prefix	Direct: ls, cat, cp	Requires: hdfs dfs -ls, hdfs dfs -cat
File Paths	Absolute (/home/user/f.txt) or relative	Always uses / as root; URI: hdfs://namenode:port/path
Access Speed	Milliseconds; ideal for small files	Seconds of latency; optimized for large files
Data Locality	Data on a single machine	Data distributed; computation moves to data
Write Model	Random read/write, in-place editing	Write-once-read-many (WORM); no in-place edit
Permissions	POSIX permissions + ACLs	Similar to POSIX + HDFS-specific ACL support

3.5 Hadoop-Only Commands (No Linux Equivalent)

- hdfs namenode -format — Format the NameNode (WARNING: erases all metadata).
- hdfs dfsadmin -safemode enter — Manually enter safe mode.
- hdfs fsck / — Check HDFS file system health.
- hdfs balancer — Rebalance data blocks across DataNodes.
- hdfs dfs -setrep 3 /file.txt — Set replication factor for a file.

3.6 Commands NOT Available in Hadoop

Some Linux commands have no direct HDFS equivalent:

- grep — Use MapReduce or Spark for pattern matching across HDFS files.
- sed, awk — Use data processing frameworks (Pig, Hive, Spark).
- find with complex predicates — Only limited functionality via hdfs dfs -ls.
- ln (symbolic links) — Not supported in HDFS.

3.7 YARN Commands

Command	Purpose
yarn node -list	List all NodeManager nodes.
yarn node -status <nodeId>	Show detailed status of a specific node.
yarn application -list	List all running YARN applications.
yarn application -kill <appId>	Kill a running YARN application.
yarn rmadmin -refreshNodes	Refresh NodeManager list in ResourceManager.
yarn rmadmin -refreshQueues	Reload capacity scheduler queue configuration.
hadoop distcp hdfs://nn1/src hdfs://nn2/dst	Distributed copy between HDFS clusters.
hadoop version	Display installed Hadoop version.

4. Development Environment & Distributions

4.1 Hadoop Distributions

Distribution	Provider	Key Features
Apache Hadoop	Apache Foundation	Open-source, community-driven, latest features, manual setup required.
Cloudera CDP	Cloudera	Enterprise-grade; includes Impala, Hue, Cloudera Manager; strong security.
Amazon EMR	AWS	Managed service; pay-per-use; deep integration with S3 and Redshift.
Azure HDInsight	Microsoft	Managed on Azure; supports Spark, Hive, HBase; integrates with Data Lake.
Google Dataproc	Google Cloud	Fast cluster provisioning (<90 seconds); auto-scaling; BigQuery integration.

4.3 Event Stream and Complex Event Processing

4.4 Practice Exercises

Exercise 1: Namespace Operations

- Create a directory structure in HDFS.
- Set name quota of 100 files on a directory.
- Set space quota of 1 GB on a directory.
- Try exceeding quotas and observe the error behavior.

Exercise 2: Cluster ID Investigation

- Locate the VERSION file on your cluster node.
- Record the Cluster ID and explain each field.
- Compare VERSION files across different DataNodes to verify consistency.

Exercise 3: Command Practice

- Upload 10 files to HDFS.
- Change permissions on all files to 755.
- Create a backup copy using hdfs dfs -cp.
- Verify disk usage with hdfs dfs -du -h.

3. MapReduce

3.1 Introduction to MapReduce

MapReduce is a programming model for processing large datasets in parallel across a distributed cluster. It breaks computation into two distinct phases: Map (transform/filter individual records) and Reduce (aggregate/summarize results). Originally proposed by Google, Hadoop's implementation runs atop HDFS and YARN.

> 📖 **Real-Life Analogy: Imagine India conducting a census across 28 states. In the Map Phase, each state's team counts its population and outputs results like (Maharashtra, 12 crore). In the Shuffle & Sort phase, results are grouped by state. In the Reduce Phase, a central team sums all values for a national total. This is exactly how MapReduce works.**

3.2 MapReduce Detailed Flow

Phase	Description
1. Input Splitting	Dataset divided into input splits (typically 128 MB = 1 HDFS block). Each split produces one Map task.
2. Map Phase	Each mapper processes records and outputs intermediate key-value pairs. Word count example: emits (word, 1) for each word.
3. Combiner (Optional)	Mini-reducer running on mapper output on the same node. If 'Hadoop' appears 50 times, emits (Hadoop, 50) instead of 50 × (Hadoop, 1). Reduces network traffic.

Phase	Description
4. Partitioner	Routes intermediate keys to specific reducers. Default: hash(key) % numReducers — ensures the same key always goes to the same reducer.
5. Shuffle and Sort	Framework transfers mapper outputs to reducers (shuffle) and sorts by key (sort). Most network-intensive phase.
6. Reduce Phase	Each reducer processes all values for its set of keys and writes final output to HDFS.
7. Output	Results written to HDFS as part-00000, part-00001, etc. (one file per reducer).

3.3 Anatomy of a MapReduce Job Run

- Client submits job to ResourceManager (YARN).
- ResourceManager allocates a container for the ApplicationMaster (AM).
- AM negotiates resources for map and reduce tasks.
- NodeManagers launch containers for individual tasks.
- Map tasks read input from HDFS, write intermediate output to local disk.
- Reduce tasks pull data from mappers (shuffle), sort, process, and write final output to HDFS.
- AM reports completion; all resources are released.

3.4 Handling Failures

Failure Type	Detection	Recovery
Map Task Failure	JVM crash or timeout (10 min)	AM reschedules on another node (up to 4 retries).
Reduce Task Failure	JVM crash or hang	Rescheduled; may re-fetch map outputs.
ApplicationMaster Failure	Missing heartbeats	RM restarts AM in a new container; resumes from checkpoint.
NodeManager Failure	Missing heartbeats (10 min)	Tasks rescheduled; node blacklisted after repeated failures.
ResourceManager Failure	Most severe SPOF	HA with standby RM + ZooKeeper for automatic failover.

3.5 Job Scheduling

Scheduler	Description
FIFO Scheduler	Jobs execute in submission order. Simple but a large job blocks all others. Not suitable for shared clusters.
Capacity Scheduler (Default)	Cluster divided into queues, each with guaranteed capacity. Idle capacity can be borrowed by other queues. Enterprise standard.
Fair Scheduler	Dynamically balances resources among all running jobs. Supports preemption for urgent jobs.

3.6 Shuffle and Sort — Deep Dive

Map Side

- Mapper writes output to a circular in-memory buffer (100 MB default).
- At 80% full, a background thread spills to disk after partitioning and sorting.
- Combiner runs on sorted data during the spill.
- Multiple spill files are merged into a single sorted output file.

Reduce Side

- Reducer fetches its partition from every mapper over HTTP (parallel copy threads).
- Data is merged using merge sort (possible because mapper output is pre-sorted).
- Merged data is fed directly into the reduce function.

3.8 MapReduce Input/Output Formats

Format	Class	Description
Text Input	TextInputFormat	Default; key = byte offset, value = line text.
Key-Value Input	KeyValueTextInputFormat	Lines split by separator into key and value.
Sequence File	SequenceFileInputFormat	Binary key-value pairs; supports compression; ideal for chaining MapReduce jobs.
NLine Input	NLineInputFormat	Each split = N lines; useful for heavy per-line processing.
Text Output	TextOutputFormat	Default output; key-value separated by tab.
Multiple Outputs	MultipleOutputs	Write to multiple files/directories based on key or custom logic.

3.9 Real-World MapReduce Applications

Application	Map Phase	Reduce Phase	Industry Example
Word Count	Tokenize text; emit (word, 1)	Sum counts per word	Product review sentiment analysis
Log Analysis	Parse logs; extract (URL, responseTime)	Avg response time per URL	API latency monitoring
Inverted Index	Emit (word, docID) per document	Group all docIDs per word	Google's original search index
ETL Processing	Clean, transform, validate records	Aggregate by business dimension	Daily POS data at retail chains

Application	Map Phase	Reduce Phase	Industry Example
Friend Recommendation	Emit pairs of mutual friends	Count mutual connections	LinkedIn 'People You May Know'